Chapter 16 · 10

Debugging & Troubleshooting

Logs · Inspect · Exec · Stats · Common Fixes

■ Analogy

• Debugging containers is like diagnosing a car problem. docker logs = check the dashboard warning lights. docker exec = open the hood and look inside. docker stats = check the fuel gauge and engine load. docker inspect = read the full technical spec sheet. Most problems are either configuration, networking, or resource exhaustion.

Key Terms

docker logs	Fetch stdout/stderr of a container. -f to follow, --tail 100 for last 100 lines
docker exec	Run a command in a running container. -it for interactive shell
docker inspect	Returns full JSON config: IP, mounts, env vars, restart count, exit code
docker stats	Live resource usage: CPU%, memory, network I/O, disk I/O per container
docker events	Real-time stream of Docker daemon events — start, stop, die, OOMKill, etc.
Exit codes	0=clean exit, 1=app error, 137=OOMKilled (SIGKILL), 143=graceful SIGTERM
OOMKilled	Container killed by kernel for exceeding memory limit — increase --memory or fix leak
Health status	healthy / unhealthy / starting — from HEALTHCHECK. Check with docker inspect
nsenter	Linux tool to enter container's namespaces from host — useful for distroless containers
docker cp	Copy files between container and host: docker cp container:/path /host/path

Common Problems & Fixes

Symptom	Diagnosis Command	Likely Fix
Container exits immediately	docker logs ; docker inspect (check ExitCode)	App crash — fix entrypoint or config
Can't connect to service	docker network inspect; docker exec curl http://service	Wrong network, wrong port, not started
OOMKilled (exit 137)	docker inspect | grep OOMKilled	Increase --memory or fix memory leak

Slow startup	docker stats; docker events	Wait for healthcheck or add depends_on
File not found in container	docker exec ls /path; docker cp	COPY path wrong in Dockerfile
Env var not set	docker exec env | grep VAR	Missing -e flag or .env file
Volume data missing	docker volume inspect; docker exec ls /mount	Wrong mount path or volume not attached

Debugging Toolkit

```
# Follow logs in real time
docker logs -f --tail 50 myapp

# Get a shell inside the container
docker exec -it myapp bash
# For Alpine: use sh instead of bash
docker exec -it myapp sh

# For distroless (no shell): use nsenter
PID=$(docker inspect --format '{{.State.Pid}}' myapp)
nsenter -t $PID -m -u -i -n -p -- sh

# Check resource usage
docker stats --no-stream

# Full container metadata
docker inspect myapp | jq '.[0].State'

# Check why container crashed
docker inspect myapp | jq '.[0].State.ExitCode'
docker inspect myapp | jq '.[0].State.OOMKilled'

# Copy logs out of container
docker cp myapp:/var/log/app.log ./app.log

# Run a one-shot debug container on same network
docker run --rm --network mynet nicolaka/netshoot \
  curl -v http://app:8080/health
```

■ Real-World: Datadog

• Datadog's engineering blog documents their container debugging workflow: they run a 'netshoot' sidecar container on the same network as the suspect container to diagnose connectivity without needing a shell in the target. For memory issues, they use docker stats + heap dumps via docker exec to identify leaks in their Java agents.